

SLIDES WRITTEN IN MARKDOWN

An introduction to mate

BERNARDO L. ESPAÑOL / TODAY

BASICS

FROM MARKDOWN TO PDF

A deck is a single Markdown file. `mate` reads that file and hands the typesetting over to the backend (only Typst right now).
These slides are made with mate. The source is `examples/example_deck/example_deck.md`, if you want to see how any of them is written.



PARAGRAPHS, LISTS AND MATH

You can write regular paragraphs. **Bold**, *italic* and `code` work as usual.

- Bullet lists too,
  - with nested and multi-paragraph items.
- As you can see.

1. Numbered lists as well,
2. with their numbers as markers.

You can also write math using Typst:

$$\int_0^\infty e^{-x^2} \, \mathrm{d}x = \frac{\sqrt{\pi}}{2}$$

PARAGRAPHS, LISTS AND MATH

example_deck.md

```
# Paragraphs, lists and math

You can write regular paragraphs.
**Bold**, _italic_ and `code` work as usual.

- Bullet lists too,
  - with nested and multi-paragraph items.
...

You can also write math using Typst:
$$ integral_0^oo e^(-x^2) dif x = sqrt(pi) / 2 $$
```

BUILDING THE DECK

That file, as it stands, builds with:

```
mate example_deck.md
```

mate writes the PDF next to the source.

TEMPLATES

A template sets the fonts, colours, background, cover and titles, and can add commands of its own. The deck picks it in the front matter:

```
---
templates: [card]
---
```

These slides use `card`, which lives in `card.py` next to the Markdown. Change that name and the whole deck changes its look.
You can also use more than one, like `templates: [card, simple]`, to mix features from different templates. If two of them change the same thing, the first one in the list is used.

STYLING TEXT AND MATH

This is some styled text.

STYLING TEXT AND MATH

This is some styled text.

You can do different things with this.

STYLING TEXT AND MATH

This is some styled text.

You can do different things with this.

Even in math mode:

$$\int_0^\infty e^{-x^2}\,\mathrm{d}x = \left|\sqrt{\pi}\right|2$$

STYLING TEXT AND MATH

Markdown's link syntax carries the styling, with the properties in the second pair of brackets:

- `[in red][color="red"]` gives you in red,
- `[smaller][fontsize=8]` gives you smaller,
- `[spaced out][letter_spacing=0.15]` gives you s p a c e d o u t.

A pair of brackets on its own applies to the whole block, which is how the equation above was pushed to the left, and this paragraph to the right.

ADDING AN IMAGE

The commands a deck can use come from its template, and you write them as `> command : arguments`. Some, like `add image`, are in the base template:

```
> add image : "figures/logo.png", width="60%"
```



The caption under a figure is an ordinary paragraph.

CODE BLOCKS

A fenced block is rendered in monospace over a box, highlighted by language. As in a command, its options follow a `:`.

relax.py

```
def relax(x, gamma=0.5, steps=100):
    """Gradient descent with a fixed step."""
    for _ in range(steps):
        x = x - gamma * grad(x)
    return x
```

Line numbers, a title bar, the colors, the width and the theme of every syntax role are options too.

LAYOUT

REGIONS

So far everything went into `content`, the region under the title, each new block below the previous one.

- To write somewhere else, switch with `> region : "name"`.
- The `anchor` of a region is where its content starts, here the top left.
- `> vspace : 1.0` adds 1 cm of space, like the one above this list.

REGIONS

title

So far everything went into `content`, the region under the title, each new block below the previous one.

- To write somewhere else, switch with `> region : "name"`.
- The `anchor` of a region is where its content starts, here the top left.
- `> vspace : 1.0` adds 1 cm of space, like the one above this list.
- `> draw layout` outlines the regions you name, as on this slide.

content

TWO COLUMNS IN ONE LINE

`> grid` cuts the active region into named cells, and every cell becomes a region of its own.

- Cells sharing a name merge into one.
- `width_ratios` and `height_ratios` set the proportions.
- Any content command also takes a `region` argument.



TWO COLUMNS IN ONE LINE

two-columns.md

```
> grid : [["text", "fig"]], hgap=0.6, width_ratios=[1, 1.2]
> region : "text"

Everything written here lands in the left cell.

- Cells sharing a name merge into one.
...

> region : "fig", anchor="center"
> add image : "figures/logo.png", width="100%"
```

CELLS THAT SHARE A NAME MERGE

Repeat a name in the template and those cells become a single region. The top and bottom rows here span both columns.

- The template is rows by columns, with row 0 on top.
- `height_ratios` gives the middle row most of the height.
- `anchors` sets the anchor of one cell.
- A grid can split a cell of another grid.

One caption running under both columns.

CELLS THAT SHARE A NAME MERGE

Repeat a name in the template and those cells become a single region. The top and bottom rows here span both columns.

head

- The template is rows by columns, with row 0 on top.
- `height_ratios` gives the middle row most of the height.

left

- `anchors` sets the anchor of one cell.
- A grid can split a cell of another grid.

right

One caption running under both columns.

foot

ELEMENTS

LEAVING THE STACK

With `floating=True` the image leaves the stack: it sits where `pos` puts it, and `anchor` picks which point of the image goes there, its centre by default. The mate logo in the corner, for example, comes from this line:

```
> add image : "figures/logo.png", width=3.4, floating=True, pos=(4.2, -2.48)
```

Positions are in centimetres from the centre of the slide, with y pointing up. The slide is 16 × 9 cm by default.



ALMOST EVERYTHING IS AN ELEMENT

A paragraph, an image, a figure, a code block and a shape are the same kind of thing. Any of them can go in a region, float at a `pos`, take an `id` or a `z_order`, and so on.



> `add` floats any element at its own `pos`, and this block of content floats the same way.

CROPPING AN IMAGE



```
> add image : ..., crop=(...)
```
keeps only a piece of the image.

STEP BY STEP

REVEALING IN STEPS

Every `> pause` opens a new step, and `||` splits a line into pieces that arrive one at a time.

REVEALING IN STEPS

Every `> pause` opens a new step, and `||` splits a line into pieces that arrive one at a time. Like this one,

REVEALING IN STEPS

Every `> pause` opens a new step, and `||` splits a line into pieces that arrive one at a time. Like this one, and this one.

REVEALING IN STEPS

Every `> pause` opens a new step, and `||` splits a line into pieces that arrive one at a time. Like this one, and this one.

The space is reserved from the start, so nothing already on the slide moves when the rest shows up.

MODIFYING AN ELEMENT

`> modify` restyles or moves anything carrying an id, from its step onward.

This paragraph holds one tagged span and a second one.

MODIFYING AN ELEMENT

`> modify` restyles or moves anything carrying an id, from its step onward.

This paragraph holds **one tagged span** and a second one.

MODIFYING AN ELEMENT

`> modify` restyles or moves anything carrying an id, from its step onward.

This paragraph holds one tagged span and **a second one**.

MASKING AN IMAGE

```
> mask image : id, ...
```
shows a window of a tagged image.



MASKING AN IMAGE

`> mask image : id, ...`
shows a window of a tagged image.
`> unmask image : id`
brings it back.



GROUPING CONTENT IN A FRAGMENT

A `markdown fragment` fence takes a whole run of content and treats it as one block. Give it an `id` to point at it later, a `region` to send it somewhere else, or any span property to restyle all of it at once.

This fragment went to the other column, in one colour.

- including this list,
- and this line.

REWRITING A BLOCK IN PLACE

An `overwrite` fence redraws a tagged block inside its own box, so nothing around it moves.

The first take on the idea, tagged with an id.

This line sits underneath and stays where it is.

REWRITING A BLOCK IN PLACE

An `overwrite` fence redraws a tagged block inside its own box, so nothing around it moves.

The replacement, drawn inside the box of the original.

This line sits underneath and stays where it is.

ONE VARIANT PER STEP

An `alternate` block puts one variant per step in the same slot, as tall as the tallest of them. Use it to step through versions of a figure.



ONE VARIANT PER STEP

An `alternate` block puts one variant per step in the same slot, as tall as the tallest of them. Use it to step through versions of a figure.



MIXING FRAGMENTS AND OVERWRITES

Everything so far can be mixed, and the layout holds. For example, a fragment reveals itself step by step and is then replaced all at once.
A fragment reveals itself gradually,

MIXING FRAGMENTS AND OVERWRITES

Everything so far can be mixed, and the layout holds. For example, a fragment reveals itself step by step and is then replaced all at once.
A fragment reveals itself gradually,
picking up a second line,

MIXING FRAGMENTS AND OVERWRITES

Everything so far can be mixed, and the layout holds. For example, a fragment reveals itself step by step and is then replaced all at once.

A fragment reveals itself gradually,
picking up a second line,
and then a third.

MIXING FRAGMENTS AND OVERWRITES

Everything so far can be mixed, and the layout holds. For example, a fragment reveals itself step by step and is then replaced all at once. And one overwrite replaces the three of them, inside their box.

PYTHON

DRAWINGS FROM PYTHON



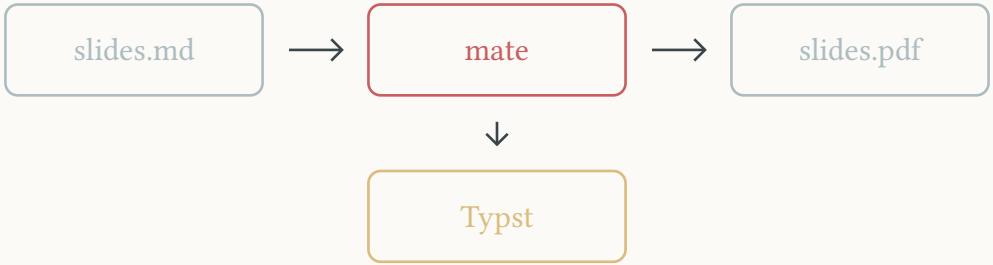
The drawing that opened this deck came from a file like this one: a `Figure` built with the Python API and added with `> add mate figure`. The slide takes the elements themselves, not an image. Colour names from the deck work inside it, and `> modify` can change anything in it that has an id.

A HIDDEN LAYER IN A FIGURE



An element built with `set_hidden(True)` holds its box and draws nothing. `> reveal` brings it in and `> hide` takes it out again, and the rest of the drawing stays where it was.

A HIDDEN LAYER IN A FIGURE



An element built with `set_hidden(True)` holds its box and draws nothing. `> reveal` brings it in and `> hide` takes it out again, and the rest of the drawing stays where it was.

SHAPES FROM A PYTHON BLOCK

A `python mate` block runs in the deck, with `self` bound to the presentation:

example_deck.md

```
for i, name in enumerate(["accent1", "accent2", "dark_gray"]):
    x = -4.6 + i * 4.6
    self.add_element(Circle(0.5, pos=(x, -2.1), fill_color=name))
    self.add_element(Text(name, pos=(x, -2.95), anchor="center"))
```



accent1



accent2



dark_gray

CONFIGURATION

THE FRONT MATTER

example_deck.md

```
---
templates: [card]
colors:
  accent1: "#b4553c"
  accent2: "#b8891a"
---
```

The front matter of this file.

- `templates` picks the look, the `card` file next to this one.
- `config` overrides any configuration key.
- `colors` adds palette entries or re-tints the existing ones by name.

ADJUSTING THE SLIDE COUNT

`> adjust slide count` moves the slide counter. It is useful when a change is long or complex, and duplicating the slide is simpler than using pauses and overwrites. With `> adjust slide count : -1` on the copy, it gets the same number as the original.

A first take on the idea.

ADJUSTING THE SLIDE COUNT

`> adjust slide count` moves the slide counter. It is useful when a change is long or complex, and duplicating the slide is simpler than using pauses and overwrites. With `> adjust slide count : -1` on the copy, it gets the same number as the original.

A second take, rewritten by hand. This slide and the one before carry the same number, and the deck's total counts them once.

FROM THE COMMAND LINE

`mate --help` lists every option. Some of them:

- `mate --info example_deck.md` lists what this deck can name: the templates, every config key with its value, the palette, the commands, the regions and the fonts that resolve.
- `mate example_deck.md --out build` writes the output to `build/`.
- `mate example_deck.md --png` writes one PNG per page.
- `mate example_deck.md --warn` builds with every check on: regions that overflow, boxes that collide and widow lines (useful for letting an LLM check a deck without looking at every slide).
- `mate --llm-docs` prints the docs written for LLMs.

I hope you find `mate` useful.
Suggestions and bug reports are welcome.

github.com/bersp/mate